



Sukkur Institute of Business Administration University
Department of Computer Science

Software Quality Engineering
BS – V (Software Engineering)
Fall 2026

Instructor: Dr Khursheed Ali

LAB # 004

Submission Profile

Name: Muhammad Anas Qadri

CMS ID: 053-24-0016

Semester: '5'

Section: 'A'

Date: 01 – September – 2026

Table of Contents

GitHub repository URL	3
docs/functional-requirments.md with 6-8 functional requirements for gradebook.....	3
docs/test-plan.md following IEEE 829 structure.....	4
docs/test-cases.md with 12 fully specified test cases	5
docs/rtn.md traceability matrix with no untraced requirements.....	5
Execution results with at least one linked defect issue for any failure found.....	6

GitHub repository URL

<https://github.com/anasqadri-dev/software-quality-engineering>

docs/functional-requirements.md with 6-8 functional requirements for gradebook

<https://github.com/anasqadri-dev/software-quality-engineering/blob/main/LABs/LAB%204/sqe-gradebook/docs/functional-requirements.md>

Functional Requirements

Requirement ID	Requirement
REQ-1	The system shall allow a valid score between 0 and 100 to be added to a student.
REQ-2	The system shall reject scores below 0 or above 100.
REQ-3	The system shall calculate the average of all recorded scores correctly.
REQ-4	The system shall handle a student with no recorded scores without crashing.
REQ-5	The system shall prevent duplicate student IDs.
REQ-6	The system shall compare student names without treating uppercase and lowercase letters as different.
REQ-7	The system shall correctly handle the minimum score boundary of 0 and the maximum score boundary of 100.
REQ-8	The system shall convert numeric scores to the appropriate grade letter, including scores at grade boundaries.

docs/test-plan.md following IEEE 829 structure

<https://github.com/anasqadri-dev/software-quality-engineering/blob/main/LABs/LAB%204/sqe-gradebook/docs/test-plan.md>

GradeBook Test Plan

1. Introduction

This test plan defines the testing approach for the GradeBook project. The project is a Java-based grade management library centered on the `Student` class, which stores a student's name, ID number, and scores. The purpose of testing is to verify that score management and student information functions behave correctly, handle invalid inputs safely, and maintain data integrity. Testing will also verify that previously fixed defects do not reappear.

2. Test Items

The primary test item is the Java `Student` class located at `src/gradebook/Student.java`.

The main functionality under test includes:

- Creating a `Student` object with a name and ID number.
- Adding valid scores using `addScore()`.
- Rejecting scores outside the range 0-100.
- Retrieving the student's name using `getStudentName()`.
- Retrieving the number of recorded scores using `getScoreCount()`.
- Handling student score data consistently after valid and invalid operations.

3. Features to be Tested

The following features will be tested:

- Student object creation with valid name and ID.
- Student name retrieval.
- Student ID storage and student identification behavior.
- Adding valid scores from 0 to 100.
- Rejecting negative scores.
- Rejecting scores greater than 100.
- Maintaining the correct score count.
- Handling an empty score list safely.
- Regression testing for previously fixed defects such as duplicate student IDs, negative scores, and empty-score average handling.

4. Features Not to be Tested

A graphical user interface is excluded because GradeBook is currently implemented as a Java library and does not contain a GUI. Database integration and network communication are also excluded because the current codebase does not implement persistent storage or external services. Performance testing at large production scale is outside the scope of this lab because the project is a small educational Java codebase.

5. Approach

Testing will primarily use functional and regression testing techniques. Positive test cases will verify that valid inputs produce the expected results, while negative test cases will verify that invalid inputs are rejected correctly. Boundary-value testing will be used for score limits such as 0 and 100, as well as invalid values below 0 and above 100. Regression tests will be executed for previously fixed defects to ensure that changes do not reintroduce earlier problems.

6. Pass/Fail Criteria

The test cycle will be considered a **PASS** when at least 95% of all planned test cases pass and 100% of Critical and High severity defects are closed or have an approved resolution. No Critical defects may remain open at release. Any failed test case related to a core GradeBook function must be investigated and either fixed or formally accepted before release.

The test cycle will be considered a **FAIL** if fewer than 95% of planned test cases pass, if any Critical defect remains open, or if a core functional requirement cannot be demonstrated successfully.

7. Test Deliverables

The following testing artifacts will be produced:

- `docs/test-plan.md` containing the overall test strategy.
- `docs/test-cases.md` containing detailed test cases.
- Requirements Traceability Matrix linking requirements to test cases.
- Java test source files under the `tests/` directory.
- Test execution results and evidence where required.
- Defect reports for any newly discovered defects.

8. Environmental Needs

Testing will be performed on a Windows development environment using Java 17 or later. The project source code is located under `src/gradebook/`, and test code will be maintained under the `tests/` directory. A Java compiler and command-line environment are required to compile and execute the tests. Git will be used to track changes to the source code and testing artifacts.

9. Schedule

Testing activities will be performed during the Lab 04 exercise. The test plan and requirements will be documented first, followed by detailed test cases and requirements traceability. Functional and negative tests will then be executed, followed by regression testing of previously fixed defects. Final review will confirm that the defined pass/fail criteria have been satisfied.

10. Risks and Contingencies

The small size of the current GradeBook implementation may limit the number of realistic production-level scenarios that can be tested. Changes to the `Student` class may introduce regressions in existing functionality, so regression tests will be executed after modifications. Incomplete requirements may also result in missing test coverage; therefore, requirements will be explicitly mapped to test cases through the RIM. If a required behavior is not implemented in the current codebase, it will be documented rather than silently assumed to exist.

docs/test-cases.md with 12 fully specified test cases

<https://github.com/anasqadri-dev/software-quality-engineering/blob/main/LABs/LAB%2304/sqe-gradebook/docs/test-cases.md>

ID	Title	Requirement	Preconditions	Steps	Expected	Priority	Type	Execution Result
TC-001	Add a valid score	REQ-1	A Student object exists with an empty scores list.	1. Create a Student object. 2. Call <code>addScore(100)</code> . 3. Check the score count.	The score is accepted and the score count becomes 1.	High	Positive / Functional	PASS — Score 100 was accepted and the score count became 1.
TC-002	Reject a negative score	REQ-2	A Student object exists with an empty scores list.	1. Create a Student object. 2. Call <code>addScore(-5)</code> .	<code>IllegalArgumentException</code> is thrown and the score is not added.	High	Negative / Functional	FAIL — Score -5 was rejected with <code>IllegalArgumentException</code> .
TC-003	Reject non-numeric score input	REQ-3	A score is provided as text before being passed to the <code>addScore</code> method.	1. Attempt to convert the input "ABC" using <code>Double.parseDouble("ABC")</code> . 2. Observe the conversion result.	A <code>NumberFormatException</code> is thrown and no invalid score is passed to <code>addScore()</code> .	Medium	Negative / Functional	BLOCKED — Student.java accepts double input only and provides no non-numeric score API.
TC-004	Calculate average with multiple scores	REQ-4	A Student object exists and contains scores 85, 90, and 75.	1. Add scores 85, 90, and 75. 2. Call <code>getAverage()</code> .	The returned average is 83.3.	High	Positive / Functional	PASS — Scores 85 and 90 produced an average of 87.5.
TC-005	Handle average with an empty scores list	REQ-5	A Student object exists with no scores.	1. Create a Student object. 2. Call <code>getAverage()</code> .	The method handles the empty list safely and returns 0.0 instead of throwing an exception.	High	Negative / Functional	PASS — An empty scores list returned an average of 0.0.
TC-006	Calculate average with a single score	REQ-6	A Student object exists with no scores.	1. Add score 75. 2. Call <code>getAverage()</code> .	The returned average is 75.0.	Medium	Positive / Functional	PASS — A single score of 75 produced an average of 75.0.
TC-007	Reject duplicate student IDs	REQ-6	A Gradebook can store multiple Student records.	1. Create Student (811) with ID 1000. 2. Attempt to create/add another Student. Attempt with ID 1000.	The duplicate student ID is rejected and only one student with ID 1000 remains registered.	High	Negative / Functional	PASS — Creating a second student with the same ID caused <code>IllegalArgumentException</code> .
TC-008	Compare student names case-insensitively	REQ-7	A Student exists with name "JOHN".	1. Search for the name "JOHN". 2. Compare it with the existing name "JOHN".	The names are treated as equal regardless of letter case.	Medium	Positive / Functional	BLOCKED — Student.java has no name comparison method to test.
TC-009	Accept maximum score boundary	REQ-1	A Student object exists with an empty scores list.	1. Call <code>addScore(100)</code> . 2. Check the score count.	Score 100 is accepted and the score count becomes 1.	High	Boundary / Functional	PASS — Score 100 was accepted with <code>IllegalArgumentException</code> . Default GPA is 0.0.
TC-010	Convert a mid-range score to a grade letter	REQ-8	Grade conversion functionality is available and a score of 75 is provided.	1. Pass score 75 to the grade conversion method. 2. Observe the returned grade letter.	The score is converted to the correct mid-range grade according to the defined grading scale.	Medium	Positive / Functional	BLOCKED — Student.java has no grade letter conversion functionality.
TC-012	Convert a score at a grade boundary	REQ-8	Grade conversion functionality is available and the boundary value 80 is provided.	1. Pass score 80 to the grade conversion method. 2. Observe the returned grade letter.	The score converts the grade assigned to the 80 boundary according to the defined grading scale.	High	Boundary / Functional	BLOCKED — Student.java has no grade letter conversion functionality.

docs/rtm.md traceability matrix with no untraced requirements

<https://github.com/anasqadri-dev/software-quality-engineering/blob/main/LABs/LAB%2304/sqe-gradebook/docs/rtm.md>

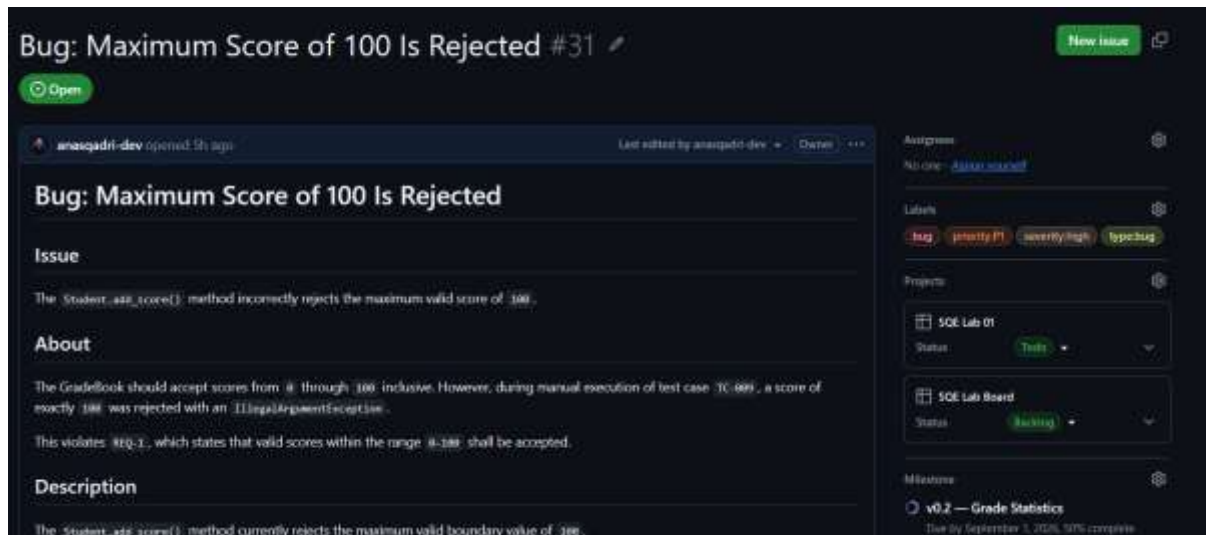
Requirement ID	Requirement	Linked Test Case IDs	Coverage Status
REQ-1	The system shall accept valid scores between 0 and 100.	TC-001, TC-009, TC-010	Covered
REQ-2	The system shall reject negative scores.	TC-002	Covered
REQ-3	The system shall reject invalid or non-numeric score input.	TC-003	Covered
REQ-4	The system shall calculate the average correctly when one or more scores are present.	TC-004, TC-006	Covered
REQ-5	The system shall handle a student with no scores without crashing.	TC-005	Covered
REQ-6	Student IDs shall be unique and duplicate IDs shall be rejected.	TC-007	Covered
REQ-7	Student name comparison shall be case-insensitive.	TC-008	Covered
REQ-8	The system shall convert numeric scores into the correct grade letters, including boundary values.	TC-011, TC-012	Covered

• Total requirements: 8
• Requirements with linked test cases: 8
• Requirements with zero linked test cases: 0
• Coverage: 100%

No requirements currently have zero linked test cases. All eight requirements have at least one corresponding test case in <code>docs/test-cases.md</code> .
This RTM was reviewed against the 12 planned test cases to ensure that the requirements are traceable to executable verification activities. The boundary, negative/extra-paths, and reversal functional scenarios are represented in the matrix.

Execution results with at least one linked defect issue for any failure found

<https://github.com/anasqadri-dev/software-quality-engineering/issues/31>



THE END